

ECAT_Learn

EtherCAT 概念

EtherCAT 数据格式

1. 外层：标准以太网帧（运输卡车）
2. 中层：EtherCAT 数据报（集装箱）
3. 核心层：EtherCAT 子报文（集装箱里的独立货箱）

EtherCAT 寻址模式

1. 设备寻址（物理寻址）
2. 逻辑寻址（过程数据寻址）
3. 总结

EtherCAT 状态机

1. Init（初始化）状态
2. Pre-Operational（预运行）状态
3. Safe-Operational（安全运行）状态
4. Operational（运行）状态
5. Bootstrap（引导）状态（可选）

状态转换流程（典型启动序列）

EtherCAT 同步

1. 核心概念：分布式时钟
2. 通信同步的实现过程
3. 任务同步：同步模式
4. 关键总结

EtherCAT SII EEPROM

1. EEPROM 物理与逻辑结构
 - 1.1. 物理特性：
 - 1.2. 逻辑数据结构：
 - 1.3. 各区域详细功能
 - 1.4. EEPROM 的访问与管理
 - 1.5. 发展趋势：ESI 文件与 EEPROM 仿真

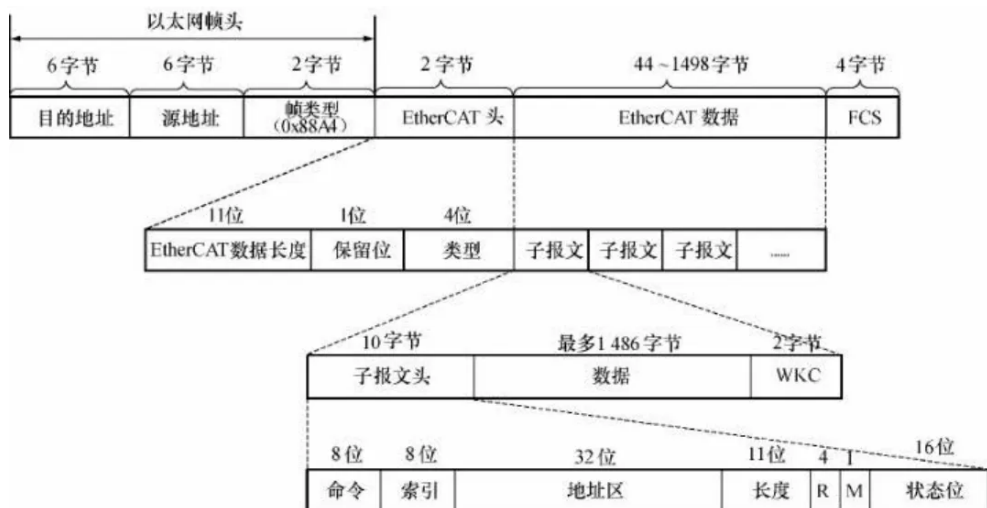
EtherCAT SM & FMMU

1. SM(同步管理器)
 - 1.1. 核心功能与作用
 - 1.2. SM 的工作原理：缓冲区管理与硬件信号量
 - 1.3. SM 的两种工作模式
 - 1.4. SM 通道配置
2. FMMU(现场总线内存管理单元)
 - 2.1. 核心功能与作用
 - 2.2. FMMU 的工作原理：位级映射
 - 2.3. FMMU 的优势与价值
3. SM 与 FMMU 的协同工作流程
4. 总结

EtherCAT应用层

1. EtherCAT COE(SDO & PDO)
 - 1.1. SDO(服务数据对象)
 - 1.1.1. 核心功能与特点
 - 1.1.2. SDO 的通信机制
 - 1.1.3. 关键概念：对象字典
 - 1.2. PDO(过程数据对象)
 - 1.2.1. 核心功能与特点
 - 1.2.2. PDO 的通信机制
 - 1.3. 协同工作流程
 - 1.4. SDO 与 PDO 的对比总结
2. EtherCAT EOE
3. EtherCAT COE
4. EtherCAT FOE

EtherCAT 概念



EtherCAT 的数据结构可以理解为三层封装，核心思想是“一个以太网帧，多个操作指令”，以实现极高的效率

1. 外层：标准以太网帧（运输卡车）

- 作用：在物理网络上传输。
- 关键字段：
 - **EtherType (类型)**：固定为 `0x88A4`。这是 EtherCAT 的“身份证”，告诉网络设备这个数据包需要由 EtherCAT 协议处理。
 - **数据域 (Payload)**：装载着真正的 EtherCAT 内容。

2. 中层：EtherCAT 数据报（集装箱）

- 作用：作为容器，包裹着所有要执行的指令。
- 关键字段：
 - **长度**：指明后面所有指令数据的总长度。

3. 核心层：EtherCAT 子报文（集装箱里的独立货箱）

这是最有用的部分。一个 EtherCAT 数据报里可以包含多个子报文，每个子报文都是一个独立的读写命令。

- **命令**：最重要的字段之一。定义了操作类型（读、写、读写）和寻址模式。
 - **逻辑寻址 (LRD, LWR, LRW)**：用于高速的过程数据交换。主站可以将所有设备的输入输出数据映射到一片连续的逻辑内存中，通过一个子报文即可同时读写多个设备，这是实现高性能的关

键。

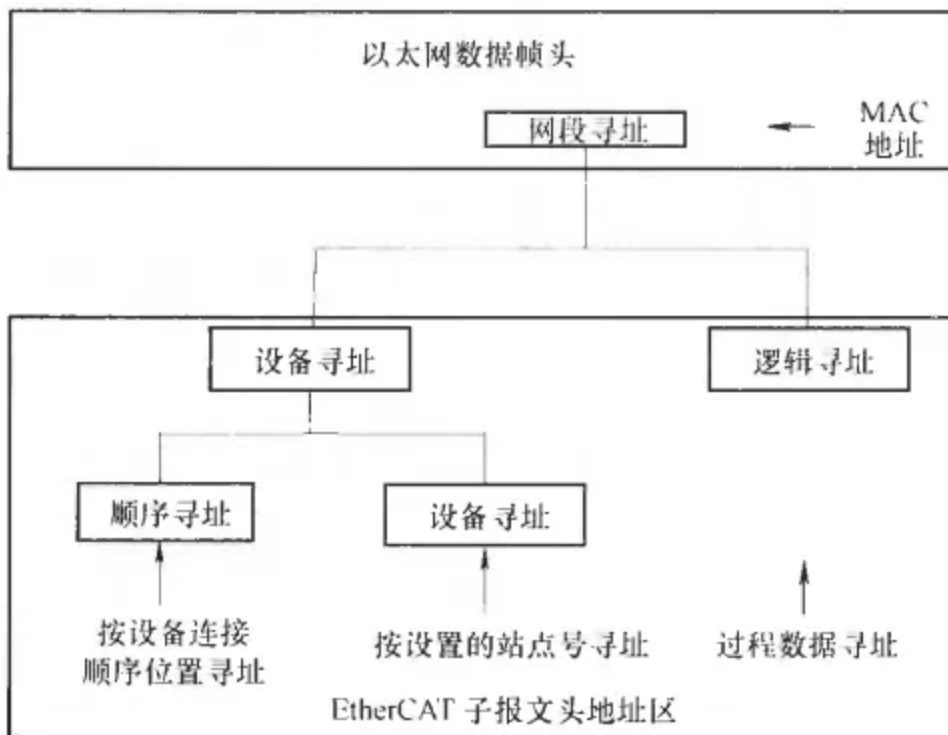
- **物理寻址 (APRD, APWR, APRW)**: 用于**设备配置和诊断**。针对某个特定从站的特定地址进行操作。
- **地址**: 32位地址, 根据命令字的寻址模式解释为逻辑地址或物理地址。
- **数据**: 要写入的数据或读回的数据。
- **工作计数器**: **极其重要的校验机制**。子报文每被一个从站成功处理一次, 这个计数器就会增加。主站通过比对返回的WKC和期望值, 可以精确判断指令是否被所有目标从站正确执行。

EtherCAT 寻址模式

EtherCAT 采用“先宏观, 后微观”的两级寻址机制, 完美匹配其主从式和“On-the-Fly”处理的工作方式。

第一级: 网段寻址 (找哪个网络)

- **方法**: 使用标准以太网帧头中的 **MAC 地址**。
- **目的**: 主站首先通过 MAC 地址找到目标 EtherCAT 网段 (整个 EtherCAT 网络在宏观上被视为一个以太网设备)。



第二级: 设备寻址 (找网段里的哪个设备) 在找到目标网段后, 使用 EtherCAT 子报文头中的 **32位地址字段** 进行精细寻址。这部分是精髓, 主要分为两种模式: 设备寻址和逻辑寻址

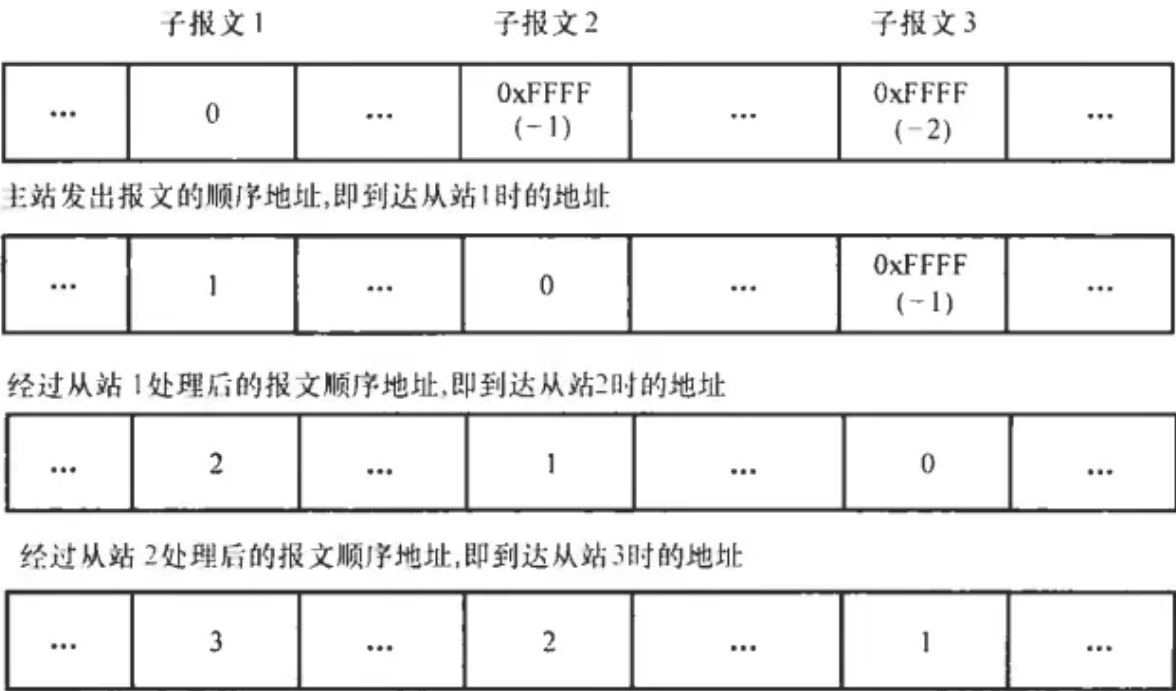
1. 设备寻址（物理寻址）

这种模式针对特定的、唯一的从站进行操作，常用于设备配置、参数设置等非周期性通信。

- 工作原理：32位地址被划分为两部分：
 - 高16位：从站设备地址（寻址65,535个设备）。
 - 低16位：该设备内部的本地地址空间（寻址64KB空间）。



- 两种子模式：
 - 顺序寻址（自动增量寻址）：
 - 地址值：使用负数（如0, -1, -2）表示从站在链路上的物理位置。
 - 工作方式：数据帧经过每个从站时，所有子报文的顺序地址自动加1。从站只处理地址变为0的那个子报文。主要用于启动阶段，为从站分配固定地址。



- 设置寻址（站地址寻址）：
 - 地址值：由主站分配或从站EEPROM预设的一个固定地址（如1000, 1001）。

- **工作方式：**与物理位置无关，主站直接使用该固定地址访问从站。这是正常运行后最常用的设备寻址方式。

2. 逻辑寻址（过程数据寻址）

这是 EtherCAT 实现高性能的“杀手锏”，用于**高速、周期性的过程数据交换**（如I/O状态、伺服控制值）。

- **核心组件：FMMU（现场总线内存管理单元）。**每个从站内的FMMU可将本地的物理内存地址映射到主站统一的、巨大的逻辑地址空间（4GB）中的任意位置。
- **工作方式：**
 - a. 主站配置每个从站的FMMU，告诉它：“你的输入数据映射到我的逻辑地址空间的A位置，你的输出数据映射到B位置”。
 - b. 主站发送一个**逻辑读写命令**的子报文，针对一片逻辑地址空间进行操作。
 - c. 所有从站同时检查这个逻辑地址是否与自己的FMMU配置匹配。如果匹配，则**并行地**从报文中提取输出数据或插入输入数据。
- **巨大优势：**主站只需**单个子报文**即可同时与**多个从站**交换数据，与设备的物理顺序无关。这极大地压缩了通信量，实现了极高的效率和确定性。

3. 总结

寻址模式	目的	特点	好比
设备寻址	配置、诊断	点对点，操作特定从站	快递员上门，把包裹交给某一户
逻辑寻址	过程数据交换	多点广播，高效并行	广播通知，所有相关住户同时收听并回应

简单来说，EtherCAT 通过 **逻辑寻址** 实现了“一对多”的并行通信，满足了实时控制对效率的极致要求；同时保留了 **设备寻址** 用于必要的点对点管理功能。

EtherCAT 状态机

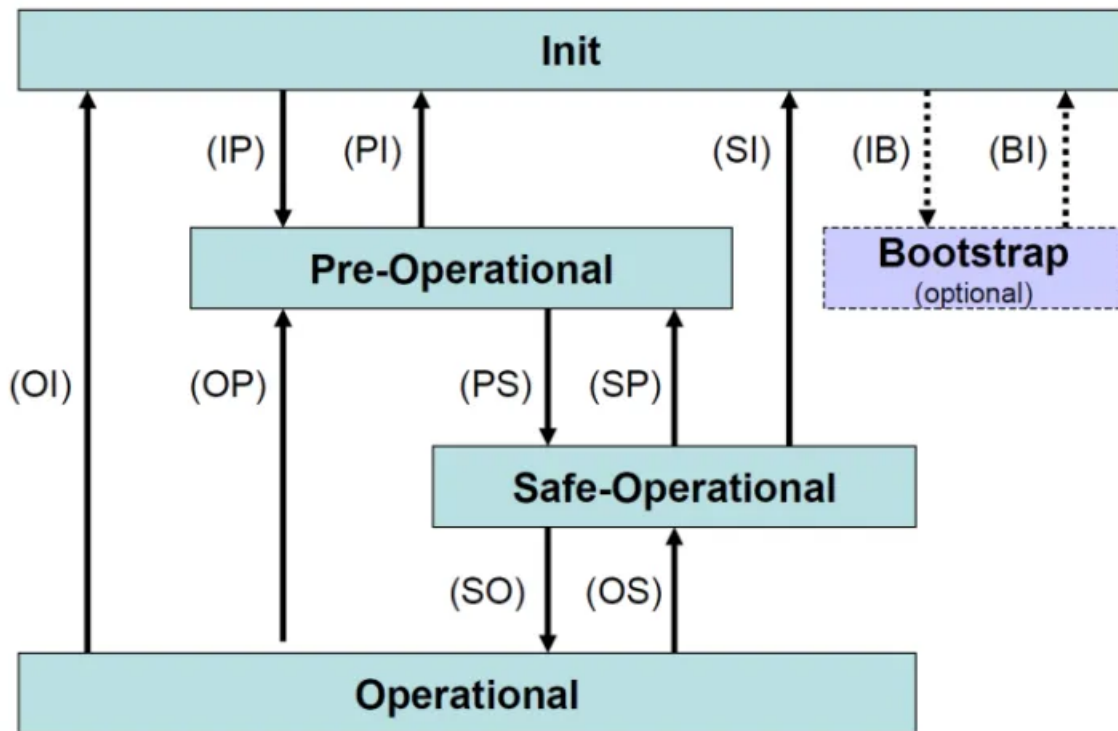
EtherCAT 状态机是确保主站和从站之间协调、有序通信的核心机制。它就像一个严格的“交通信号灯系统”，控制着从站设备可以执行哪些操作，以及何时可以执行。

理解状态机对于进行 EtherCAT 网络调试和故障诊断至关重要。当通信出现问题时，首先检查设备当前处于哪个状态，是排查问题的第一步。

基本规则：

1. **主站主导**：所有状态转换的指令都由主站发出。
2. **从站响应**：从站根据自身情况执行转换，并报告结果。
3. **顺序转换**：从初始化到完全运行，必须按顺序经历特定状态，**不能跳级**。但从高级状态返回时可以跳级。

EtherCAT 状态机 (ESM) 用于协调主从站的通讯功能：



1. Init（初始化）状态

- **状态描述**：这是设备上电或复位后的起始状态。应用层通信尚未建立。
- **允许的通信**：
 - 仅限主站通过寄存器读写来配置从站控制器（ESC）的基本参数。
 - 例如：配置从站的地址、分布式时钟参数等。
- **禁止的通信**：不能进行任何过程数据或邮箱通信。

2. Pre-Operational（预运行）状态

- **状态描述：** 邮箱通信通道被激活，但过程数据通信尚未开启。
- **允许的通信：**
 - **邮箱通信：** 主站和从站可以通过邮箱（如CoE, SoE）交换**非周期性数据**。
 - 例如：下载设备参数、配置对象字典、上传设备信息等初始化操作。
- **禁止的通信：** 不能交换周期性的过程数据（PDO）。

3. Safe-Operational（安全运行）状态

- **状态描述：** 过程数据通信开始，但设备输出被强制置于“安全状态”（如伺服驱动器不输出扭矩）。
- **允许的通信：**
 - **邮箱通信：** 仍然可用。
 - **过程数据通信：** 主站可以**读取**从站的输入数据（如传感器信号），但从站**不会执行**主站发送的输出指令。这允许主站监控系统状态，但不会产生危险动作。
- **应用场景：** 这是系统投入运行前的最后安全检查阶段。

4. Operational（运行）状态

- **状态描述：** 这是设备的全功能运行状态。所有通信服务都被激活。
- **允许的通信：**
 - **邮箱通信：** 仍然可用，用于非紧急的参数修改或诊断。
 - **过程数据通信：** 输入和输出均有效。主站发送控制命令，从站执行并反馈状态。这是实现实时控制的阶段。

5. Bootstrap（引导）状态（可选）

- **状态描述：** 一个特殊状态，只与 Init 状态互相转换。
- **功能：** 主要用于通过 FoE（EtherCAT over Ethernet）协议进行**固件更新**。在此状态下，设备可以接收并烧写新的固件程序。

状态转换流程（典型启动序列）

一个典型的自动化设备启动过程如下：

1. **上电：** 从站进入 Init 状态。

2. **基本配置**：主站配置从站的地址、通信参数等。
3. **进入 Pre-Operational**：主站发送命令，从站切换至该状态。
4. **应用层配置**：主站通过邮箱通信（CoE）配置设备参数，如 PDO 映射、同步模式等。
5. **进入 Safe-Operational**：主站发送命令，从站切换至该状态。此时主站开始接收输入数据，但输出被禁用。
6. **进入 Operational**：主站确认系统安全后，发送命令进入最终状态。从站开始正常执行控制任务，系统正式运行。

EtherCAT 同步

EtherCAT 的同步机制是其实现高精度实时控制的核心，尤其适用于多轴运动控制等要求严格同步的应用。其同步体系主要由两部分组成：**通信同步**和**任务同步**

1. 核心概念：分布式时钟

EtherCAT 同步的基石是 **分布式时钟** 技术。它的核心思想是：**让网络中所有设备的本地时钟与一个参考时钟保持高度一致**，从而在所有从站节点上创建一个统一的系统时间基准。

- **参考时钟**：通常指定网络中的第一个支持DC功能的从站作为参考时钟，它为整个网络提供系统时间。
- **从时钟**：网络中其他支持DC的从站都有自己的本地时钟，称为从时钟。
- **系统时间**：一个从公元2000年1月1日开始计时的64位计时器，单位为纳秒
- **目标**：通过精确的补偿算法，消除所有从时钟与参考时钟之间的**初始偏移**、**传输延迟**和**时钟漂移**，使它们的系统时间保持一致。

2. 通信同步的实现过程

分布式时钟的同步是一个持续的过程，分为初始化和运行两个阶段。

初始化阶段（一次性校准）：此阶段在系统启动时执行，目的是补偿固定的时间偏差。

1. **传输延迟测量**：主站发送测量帧，精确计算数据帧从参考时钟传输到每个从时钟所需的物理延迟时间。
2. **初始偏移补偿**：主站计算每个从时钟与参考时钟之间的初始时间差，并将该偏移量和传输延迟值写入各从站。

3. **建立统一时间**：此后，每个从站使用公式 $\text{本地系统时间} = \text{本地硬件时钟} - \text{初始偏移量} - \text{传输延迟}$ 来计算其本地系统时间，从而与参考时钟初步对齐。

运行阶段（周期性动态补偿）：此阶段在系统运行中持续进行，目的是补偿因晶体振荡器频率微小差异导致的时钟漂移。

1. **误差检测**：主站周期性地读取参考时钟的当前系统时间并广播给各从站。
2. **闭环调整**：各从站将接收到的参考时间与自身系统时间进行比较，计算出漂移误差。随后，通过锁相环等控制算法，动态微调自身时钟的运行频率（加快或减慢），逐步消除误差，实现长期精确同步。

3. 任务同步：同步模式

通信同步（统一的时间基准）本身不是目的，最终目的是实现**应用程序任务的同步**。

在建立了统一的时间基准后，EtherCAT 定义了不同的**任务同步模式**，以决定从站应用程序的执行时机。

1. 自由运行模式

- **触发源**：从站内部的本地定时器。
- **特点**：从站的应用程序循环与 EtherCAT 主站通信循环无关。各从站之间不同步。
- **精度**：低（毫秒级），取决于本地定时器的精度。
- **应用场景**：对实时性和同步性要求不高的简单 I/O 控制。

2. SM同步模式（同步于报文）

- **触发源**：接收到主站发送的周期性数据帧时产生的硬件中断。
- **特点**：从站的应用程序循环由数据到达事件触发。但由于数据帧到达各从站的时间存在先后（传输延迟），各从站的动作在时间上存在误差。
- **精度**：中等（微秒级）。
- **应用场景**：有一定实时性要求，但同步精度要求不苛刻的应用。

3. DC同步模式（同步于分布式时钟）

- **触发源**：由已同步的分布式时钟产生的周期性 **SYNC 信号**（如 SYNC0、SYNC1）。
- **特点与工作流程（这是精度最高的模式）**：
 - a. **数据预置**：主站提前发送过程数据帧，从站接收数据并完成所有计算准备。
 - b. **同步执行**：当预设的 SYNC 信号到来时，**网络上所有从站同时执行输出更新**（如驱动电机）和**输入锁存**（如采样传感器信号）。
- **精度**：极高（纳秒级），消除了传输延迟和计算时间抖动的影响。
- **应用场景**：高精度多轴运动控制、机器人、飞剪、包装等要求严格同步的复杂自动化系统。

4. 关键总结

特性	自由运行模式	SM同步模式	DC同步模式
同步源	本地时钟	数据帧到达事件	分布式时钟SYNC信号
从站间同步	否	否	是
精度	低 (ms)	中 (μs)	高 (ns)
核心优势	简单、独立	实时性较好	绝对同步、高精度

结论： EtherCAT 通过 **分布式时钟** 技术为所有设备提供了统一的时间尺度，再通过 **DC同步模式** 将此刻度转化为精准的动作触发器，从而实现了整个控制系统在亚微秒级别上的协同工作。

EtherCAT SII EEPROM

SII 全称为 **从站信息接口**，是 EtherCAT 协议规范中定义的一个标准接口。它要求每个 EtherCAT 从站设备配备一块非易失性存储器（通常为 I²C 接口的 EEPROM），用于存储该从站的**身份标识信息、配置参数和功能描述**。

没有正确配置的 SII EEPROM，EtherCAT 从站将无法被主站正常识别和配置，通信也无法建立。

1. EEPROM 物理与逻辑结构

1.1. 物理特性：

- **接口：** 通常采用标准的 I²C 串行接口。
- **容量：** 常见容量从 1 Kbit (128 Bytes) 到 64 Kbit (8 KB) 或更大，具体取决于从站功能的复杂程度。
- **寻址：** 以 **16位字 (Word)** 为基本单位进行读写。

1.2. 逻辑数据结构：

EEPROM 的存储空间被划分为多个具有特定功能的区域，其结构如下图所示：



图 3.21 EEPROM 数据布局

1.3. 各区域详细功能

1. **ESC 配置区（字地址 0x000 – 0x007）** 这是 EEPROM 中最关键的区域。在从站设备上电或复位时，ESC 会自动读取这前 8 个字的内容，并将其加载到内部相应的配置寄存器中。

- **主要内容：**

- **PDI Control** (0x0140): 配置物理设备接口类型（如 8/16 位微控制器、SPI、数字 IO 等）。
- **Distributed Clock Control** : 配置分布式时钟功能是否使能。
- **Station Alias** (0x0012): 配置从站的别名地址。

- **校验和：** 最后一个字 (0x007) 是前 7 个字的校验和。ESC 会进行校验，如果失败，则视为 EEPROM 数据无效，从站可能无法进入预运行状态。

2. **设备标识区（字地址 0x008 – 0x00F）** 这是主站用来识别设备的“身份证”。

- **Vendor ID:** 设备制造商的唯一标识符，由 ETG 分配。
- **Product Code:** 制造商为该产品分配的唯一代码。
- **Revision Number:** 产品的硬件或软件版本号。
- **Serial Number:** 每个设备的唯一序列号。主站通过扫描网络，读取每个从站的这些信息，并与主站工程中导入的 **ESI (EtherCAT Slave Information) XML 文件** 进行匹配，从而正确识别设备并加载对应的配置。

3. **分类信息区（字地址 $\geq 0x040$ ）** 该区域以灵活的数据结构存储设备的详细功能描述，采用“类型–长度–数据”的格式。

- 常见类别：
 - **Type 0x0000: Strings**: 存储设备名称、供应商名称等文本信息。
 - **Type 0x0001: General**: 设备的一般信息，如支持的邮箱协议、PDO数量等。
 - **Type 0x0002: FMMU**: 描述FMMU单元的配置。
 - **Type 0x0003: SyncManager**: 描述SyncManager通道的配置。
 - **Type 0x0004: TxPDO / RxPDO**: 定义发送和接收过程数据对象的结构和映射。
 - **Type 0x0010: Mailbox**: 邮箱协议配置（如CoE、FoE）。

1.4. EEPROM 的访问与管理

1. **访问方式**: EEPROM 可以通过两种方式访问：
 - **ESC 自动加载**: 上电时，ESC 自动读取 ESC 配置区。
 - **主站或本地微控制器访问**: 通过读写 ESC 内部的 **EEPROM 接口寄存器**（地址范围通常为 0x0500 – 0x0510）来执行读、写、重载等操作。主站常用此方式在初始化阶段读取设备标识和分类信息。
2. **访问权限仲裁**: 为避免冲突，ESC 提供了寄存器位来仲裁 EEPROM 的访问权，决定当前是由 **EtherCAT 主站** 还是由 **本地微控制器** 来控制 EEPROM 接口。

1.5. 发展趋势: ESI 文件与 EEPROM 仿真

- **ESI 文件**: 现代 EtherCAT 主站配置工具（如 TwinCAT）更倾向于使用标准化的 **ESI (XML) 文件** 来描述从站。主站直接将 ESI 文件中的配置信息下载到从站的 RAM 中运行，而非完全依赖读取 EEPROM 中的分类信息区。这使得配置更加灵活和易于管理。
- **EEPROM 仿真**: 为了节省成本和PCB空间，许多现代微控制器方案（如先楫半导体的HPM5E和 HPM6E系列MCU）支持用片内Flash的一部分来模拟外部EEPROM的功能，即 **EEPROM 仿真**。这需要在软件初始化时将必要的 SII 数据写入仿真的存储区。

EtherCAT SM & FMMU

在一个 EtherCAT 系统中，主站需要与多个从站高效、安全地交换数据。

- **SM** 如同 **交通警察**，负责管理数据交换的“车道”，确保数据一致性，防止主站和从站应用同时读写同一块内存区域，造成数据错乱（竞争条件）。
- **FMMU** 如同 **地址翻译官**，负责将主站看到的“统一大地址空间”（逻辑地址）翻译成每个从站内部的“小地址空间”（物理地址），从而实现高效、灵活的数据映射。

1. SM(同步管理器)

1.1. 核心功能与作用

SM 是 EtherCAT 从站控制器内部的一个硬件单元，其主要作用是**管理主站与从站本地应用（微处理器）对 ESC 内部共享内存的访问**。它确保了数据交换的一致性和安全性。

- **一致性**：保证主站或从站应用读取到的是一份完整、准确的数据副本，而不是正在被修改的、不完整的数据。
- **安全性**：防止未经授权的或不合时宜的访问，避免数据冲突。

1.2. SM 的工作原理：缓冲区管理与硬件信号量

SM 将一段连续的 ESC 内存空间划分为一个或多个**缓冲区**，并通过硬件机制控制对这些缓冲区的访问权限。

- **缓冲区**：一块预定义的内存区域，用于存放特定方向的数据（如输入数据或输出数据）。
- **访问控制**：SM 为每个缓冲区维护一个状态机（如“空闲”、“主站已写”、“从站已读”）。当一方（如主站）完成对缓冲区的写入操作后，SM会**锁定该缓冲区，并生成一个硬件中断事件通知另一方**（如从站应用）。另一方在处理完数据后，会释放缓冲区，从而允许下一次写入。

1.3. SM 的两种工作模式

根据数据交换的需求，SM 可配置为两种基本模式：

模式	描述	适用场景	比喻
邮箱模式	握手式通信 。采用 单缓冲区 。发送方必须等待接收方读取并释放缓冲区后，才能发送新数据。确保数据不丢失。	非周期性数据 ，如参数配置、故障诊断、固件更新（CoE, FoE）。	像 寄信 ，必须等邮差取走你投递的信，才能投递下一封。

缓冲区模式	流水线式通信。通常采用双缓冲区或三缓冲区。允许发送方连续写入，接收方连续读取，利用多个缓冲区交替工作，极大提高效率。	周期性过程数据，要求高实时性和高吞吐量的 I/O 数据交换。	像流水线作业，工位A在加工第2个零件时，工位B可以同时处理第1个零件。
-------	--	--------------------------------	-------------------------------------

1.4. SM 通道配置

一个 ESC 通常提供多个 SM 通道（如 4 个或 8 个）。典型的配置如下：

- SM0：邮箱通道，用于主站到从站的邮箱通信（如发送命令）。
- SM1：邮箱通道，用于从站到主站的邮箱通信（如发送响应）。
- SM2：缓冲区通道，用于过程数据输出（主站 -> 从站）。
- SM3：缓冲区通道，用于过程数据输入（从站 -> 主站）。

主站在初始化阶段会配置每个 SM 通道的**起始地址**、**长度**和**工作模式**

2. FMMU(现场总线内存管理单元)

2.1. 核心功能与作用

FMMU 是 ESC 内部的另一个硬件单元，其核心功能是实现**逻辑寻址**。它将主站定义的**逻辑地址空间**映射到从站的**物理地址空间**。

- **逻辑地址**：一个巨大的、连续的地址空间（4GB），由主站统一规划。主站将所有从站需要交换的数据（如所有输入、输出信号）都放在这个虚拟的“大内存”中。
- **物理地址**：每个从站 ESC 内部 RAM 的实际地址。

2.2. FMMU 的工作原理：位级映射

FMMU 的强大之处在于它支持**位级**的灵活映射。主站可以配置一个 FMMU 单元，指定：

- **逻辑起始地址**：映射在全局逻辑地址空间中的起始字节地址。
- **逻辑起始位**：从上述逻辑起始地址字节内的第几位（0-7）开始映射。
- **长度**：要映射的数据区域的总体大小，以字节为单位。这个长度决定了映射所跨越的逻辑地址范围。

- **逻辑停止位**：在映射的最后一个字节内，映射停止的位位置。通过 **长度**、**逻辑起始位** 和 **逻辑停止位** 共同确定了需要映射的**精确的位序列**。
- **物理起始地址**：映射到本从站 ESC 内部物理内存的起始字节地址。
- **物理起始位**：从上述物理起始地址字节内的第几位开始存放数据。
- **操作类型**：数据的映射方向（1：只读，即输入；2：只写，即输出；3：读写）。

表 2.3 FMMU 配置示例

FMMU 配置寄存器	数 值
数据逻辑起始地址	0x00014711
数据长度（字节数，按跨字节计算）	2
数据逻辑起始位	3
数据逻辑终止位	0
从站物理内存起始地址	0x0F01
物理内存起始位	1
操作类型（1：只读，2：只写，3：读写）	2
激活（使能）	1

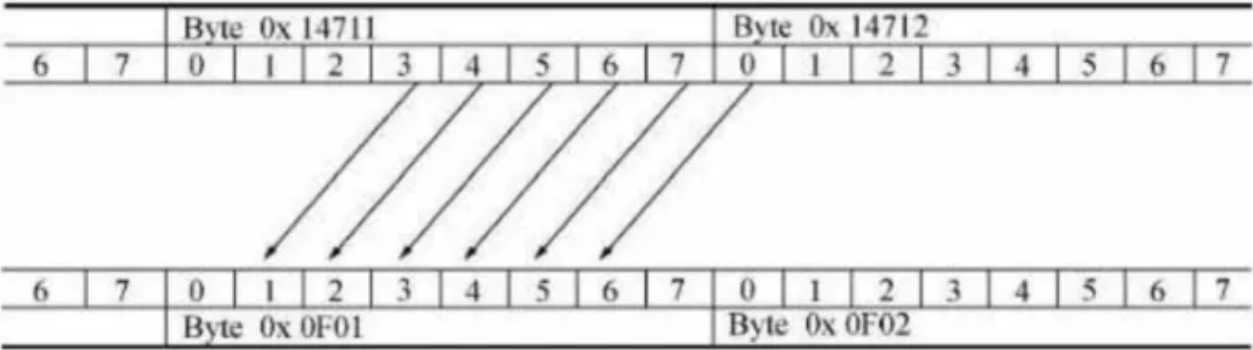


图 2.18 FMMU 映射举例

2.3. FMMU 的优势与价值

- **极高的带宽利用率**：主站只需构造一个大型 EtherCAT 数据帧，其中包含多个逻辑地址连续的子报文。该帧经过网络时，每个从站的 FMMU 会同时从帧中提取属于自己的数据（输出），或将数据插入帧中（输入）。这避免了为每个从站单独发送数据帧的开销。

- **灵活的拓扑结构：**数据顺序与从站的物理连接顺序无关。主站可以像访问一块大内存一样访问所有 I/O 点，极大简化了应用程序的设计。

3. SM 与 FMMU 的协同工作流程

让我们通过一个**周期性过程数据通信**的例子，看看 SM 和 FMMU 是如何协同工作的：

1. **主站构建数据帧：**主站应用程序周期性地准备要发送的输出数据（如控制电机的指令值），这些数据位于逻辑地址空间中。主站协议栈根据 FMMU 配置，生成一个包含“逻辑写”命令的 EtherCAT 帧。
2. **数据帧遍历网络：**
 - 数据帧到达第一个从站。该从站的 FMMU 检查帧中的逻辑地址。

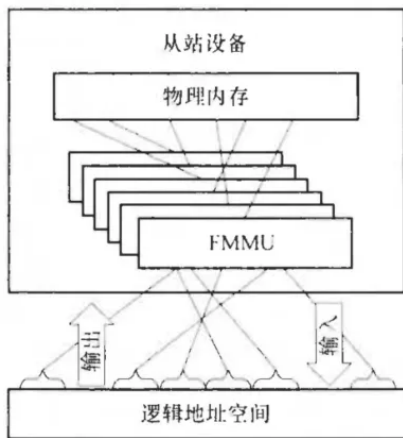


图 2-21 现场总线内存管理单元

- **FMMU 工作：**如果逻辑地址落在其配置的映射范围内，FMMU 会**即时地**将数据从帧中提取出来，写入 ESC 内部 RAM 的对应物理地址（该地址由 SM2 管理）。
 - **SM 工作：**当数据写入完成后，**SM2 的硬件被触发**，可能会产生一个中断给本地微处理器，告知“新的输出数据已就绪”。
3. **从站处理数据：**从站的微处理器响应 SM2 的中断，从 SM2 管理的缓冲区中读取输出数据，并驱动执行机构（如电机）。
 4. **输入数据采集：**同时，从站微处理器将采集到的输入数据（如电机实际位置）写入由 SM3 管理的缓冲区。
 5. **数据帧返程：**
 - 数据帧继续传播到最后一个从站后折返。
 - 在返程再次经过该从站时，其 FMMU 会将 **SM3 缓冲区中的输入数据**插入到数据帧对应的逻辑地址位置。

6. **主站接收处理**：数据帧返回主站，主站协议栈解析帧中的输入数据，更新到应用程序的逻辑地址空间，完成一个通信周期。

4. 总结

特性	同步管理器 (SM)	现场总线内存管理单元 (FMMU)
核心职责	数据一致性管理 (“何时”访问)	地址映射 (“何处”访问)
工作层面	管理单个从站内部的数据缓冲区访问	管理整个网络层面的逻辑到物理地址转换
主要目的	保证数据安全、一致，避免冲突	提高带宽利用率，实现灵活数据布局
比喻	交通警察	地址翻译官

简而言之，FMMU 决定了数据“放在哪里”，而 SM 决定了数据“何时可被安全读写”。它们共同将 EtherCAT 网络打造成一个高效、确定性的实时通信系统，这是其性能远超传统以太网的关键所在。

EtherCAT应用层

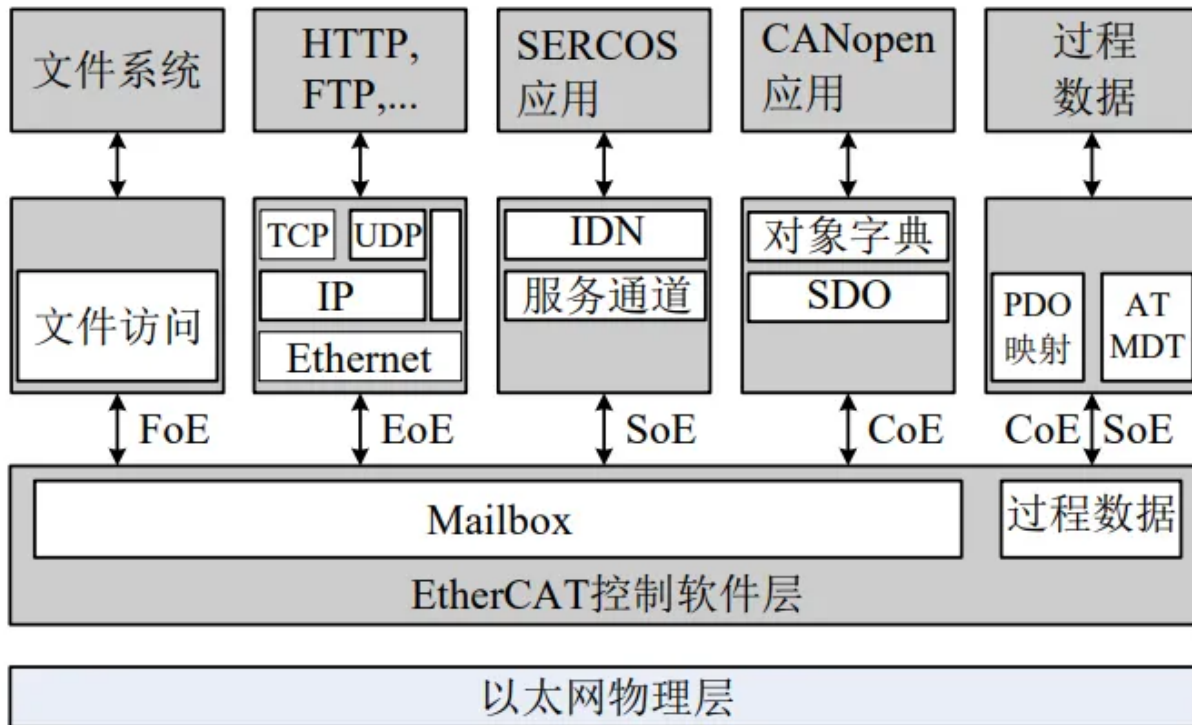


图 1.3 EtherCAT 协议栈

1. EtherCAT COE(SDO & PDO)

CoE 的全称是 CANopen over EtherCAT。

- **本质**：它是一种将成熟的 CANopen 设备行规和应用层协议映射到高性能的 EtherCAT 传输系统上的协议。
- **目标**：实现设备互操作性。这意味着，一个遵循 CoE 标准的伺服驱动器（例如，遵循 CiA 402 行规）可以被任何支持 CoE 的 EtherCAT 主站识别和控制，无论其制造商是谁。
- **核心**：CoE 的核心是 **对象字典**，而 SDO 和 PDO 是访问和操作这个对象字典的两种基本机制。

把 EtherCAT 从站设备想象成一个复杂的智能家电（比如一台高级洗衣机）。

- **SDO** 就像是你通过手机 APP 配置洗衣机：设置洗涤模式、水温、转速等。这些操作是**非周期性**的，只在需要改变时才发生，传输的是**配置和参数数据**。
- **PDO** 就像是洗衣机运行过程中，控制面板实时显示的当前水温、剩余时间，以及你随时可以按下的**启动/暂停按钮**。这些是**周期性的、实时的**数据交换，传输的是**控制命令和状态反馈**。

1.1. SDO(服务数据对象)

1.1.1. 核心功能与特点

- **用途**：用于非周期性的通信。主要任务是访问从站的对象字典，进行设备配置、参数设置、故障诊断、固件更新等。
- **通信模式**：客户端-服务器模型（请求-响应）。主站是客户端，从站是服务器。主站发送一个 SDO 请求，从站必须返回一个 SDO 响应，确认操作完成。这是一种可靠的、有确认机制的通信。
- **实时性**：非实时或软实时。对时间不敏感，允许一定的延迟。通信通常在系统启动初始化阶段或运行时偶尔发生。
- **数据量**：可以传输相对较大的数据块（通过分段传输）。

1.1.2. SDO 的通信机制

SDO 通信通过 邮箱数据通道 进行，由 同步管理器 管理（如 SM0 和 SM1），确保数据在传输过程中不会冲突。

- **SDO 写**：主站要修改一个参数（如修改驱动器最大速度 `0x607F`）。
 - a. 主站构造一个 SDO 写请求帧，通过邮箱发送到从站。
 - b. 从站收到后，解析请求，将新值写入对象字典的相应位置。
 - c. 从站返回一个 SDO 写响应帧，告知主站“写入成功”。
 - d. 主站收到成功响应后，才知道参数已修改。
- **SDO 读**：主站想读取一个参数（如读取设备序列号 `0x1018`）。
 - a. 主站构造一个 SDO 读请求帧。
 - b. 从站收到后，从对象字典中读取请求的数据。
 - c. 从站返回一个 SDO 读响应帧，其中包含请求的数据。
 - d. 主站从响应中获取数据。

1.1.3. 关键概念：对象字典

对象字典是一个有序的集合，包含了描述设备的所有参数。每个参数被称为一个 **对象**，通过一个 16 位的索引 和一个 8 位的子索引 来唯一寻址。

- 索引 `0x6040`：控制字 – 用于控制驱动器状态（如启动、停止）。
- 索引 `0x6064`：位置反馈 – 读取驱动器的实际位置。
- 索引 `0x60B1`：最大电流 – 配置驱动器的电流限制。

SDO 就是用来读写这些索引对应的值的。

1.2. PDO(过程数据对象)

1.2.1. 核心功能与特点

- **用途：**用于周期性的实时数据交换。传输控制回路中需要不断刷新的数据，如电机转矩指令、实际位置反馈、数字量 I/O 状态等。
- **通信模式：生产者-消费者模型。**
 - **生产者：**数据的发送方（如主站生产控制指令，从站生产状态反馈）。
 - **消费者：**数据的接收方（如从站消费控制指令，主站消费状态反馈）。
 - 数据一旦发出，可以被多个消费者接收（如广播）。这是一种高效的、无确认机制的通信。
- **实时性：硬实时。**数据必须在严格的周期内送达，否则可能导致控制失效。这是 EtherCAT 高性能的体现。
- **数据量：**通常较小，只包含最关键的实时数据，以最小化通信延迟。

1.2.2. PDO 的通信机制

PDO 通信通过 过程数据通道 进行，由 同步管理器 管理（如 SM2 和 SM3），并利用 FMMU 实现高效映射。

- **数据传输：**PDO 数据被直接映射到 EtherCAT 帧中，在每一个通信周期内，主站和从站之间自动交换，不需要单独的请求和响应帧。
- **PDO 映射：**这是 PDO 的核心配置步骤。它定义了哪些对象字典中的参数应该通过 PDO 进行周期性传输。
 - **RxPDO（接收 PDO，从站视角）：**定义从站接收哪些主站发来的实时数据（如控制字 0x6040，目标位置 0x607A）。
 - **TxPDO（发送 PDO，从站视角）：**定义从站发送哪些实时数据给主站（如状态字 0x6041，实际位置 0x6064）。

示例：一个简单伺服驱动的 PDO 映射

PDO 类型	映射的对象	功能说明
RxPDO	0x6040:00 (控制字)	主站发送命令，控制驱动器启停
	0x607A:00 (目标位置)	主站发送目标位置指令

TxPDO	0x6041:00 (状态字)	从站反馈当前状态（如就绪、故障）
	0x6064:00 (实际位置)	从站反馈电机实际位置

在运行时，主站只需在每个周期将 **0x6040** 和 **0x607A** 的值更新到对应的逻辑地址，EtherCAT 主站协议栈会自动将其打包进帧并发送。从站也会自动将 **0x6041** 和 **0x6064** 的值放入帧中返回给主站。整个过程非常高效。

1.3. 协同工作流程

在一个典型的 EtherCAT 系统启动和运行过程中，SDO 和 PDO 协同工作：

1. 上电/初始化阶段（使用 SDO）：
 - 主站读取从站的标识信息（如厂商ID、产品代码），并加载对应的配置文件。
 - 主站使用 SDO 配置从站的通信参数，其中最关键的一步就是**配置 PDO 映射**（即告诉从站哪些数据需要周期性交换）。
 - 主站使用 SDO 配置设备的工作模式、极限参数等。
2. 进入运行阶段（切换到 PDO）：
 - 主站发送命令，让从站进入**运行状态**。
 - 此后，主从站之间绝大部分通信都通过高效的 **PDO 周期性进行**，实现实时控制。
3. 运行中（SDO 作为补充）：
 - 如果需要动态改变 PDO 映射（如切换控制模式）、修改参数或读取诊断信息，主站会再次使用 SDO，但这类操作不会影响 PDO 的周期性通信。

1.4. SDO 与 PDO 的对比总结

特性	服务数据对象	过程数据对象
通信目的	配置、参数化、诊断	实时控制
通信模型	客户端-服务器（请求/响应）	生产者-消费者（发布/订阅）
实时性	非实时/软实时	硬实时
周期性	非周期性（事件驱动）	周期性（固定周期）

数据可靠性	高（有确认机制）	依靠周期刷新（无确认）
数据量	可大可小（分段传输）	小（仅关键数据）
通信通道	邮箱（SM0, SM1）	过程数据区（SM2, SM3）
依赖关系	PDO 的映射关系需要通过 SDO 来配置	PDO 的实时运行依赖于 SDO 的前期配置

2. EtherCAT EOE

3. EtherCAT COE

4. EtherCAT FOE